



SISTEMAS DE BANCO DE DADOS 2



AULA 3

Revisão de SQL básico - *Structured Query Language* **MySQL**

Vandor Roberto Vilardi Rissoli



APRESENTAÇÃO

- Linguagem SQL
- Linguagem de Definição de Dados
- Linguagem de Manipulação de Dados
- Referências



Linguagem em Banco de Dados

De forma geral, os Banco de Dados Relacionais (SGDBs) implementam o padrão SQL e acrescentam características específicas ao padrão.

Cada SGBD procura incluir elementos que o possam diferenciar dos concorrentes e implementam também algumas possibilidades procedurais da SQL, por exemplo, a *Transact SQL* para o **SQL SERVER** e o *PL/SQL* para a **ORACLE** entre outros SGBDs.

Este material apresentará a SQL tão portável quanto possível, mas as instruções nele apresentadas poderão ser implementadas no SGBD **MySQL**.



Structured Query Language - SQL

A SQL surgiu no início da década de 70, por uma **iniciativa da IBM**. Ela tornou-se a linguagem mais popular para acesso aos bancos de dados, juntamente com a difusão dos SGBDs Relacionais.

Vários “dialetos” surgiram rapidamente e a ANSI padronizou o SQL, surgindo em 1986 SQL-86 e outros padrões que seguem (89, 92, SQL-101,...) em evolução até hoje.



Segue ANSI SQL: 2011

- *Common Table Expressions*
- *Window Functions*
- Tipos de dados e **JSON**
- Flexível pelo **sql_mode**, ...

Structured Query Language - SQL

CARACTERÍSTICAS DAS LINGUAGENS

A linguagem SQL é **declarativa** e os SGBDs Relacionais a têm como padrão. Suas características são originárias da **Álgebra Relacional**.

- Linguagem **Declarativa**: (ou **Não** Procedural): Linguagem que descreve O QUE se deseja realizar, SEM se preocupar em dizer COMO será feito. O sistema é que deverá determinar a forma de como fazer (não se descreve ou conhece como o sistema realizará tal processamento);
- Linguagem **Procedural**: Linguagem que fornece uma descrição detalhada de COMO um processamento será realizado, operando sobre um registro ou uma unidade de dados de cada vez.



Structured Query Language - SQL

A expressão **Programação Declarativa** é usada para discernir da programação praticada por meio das Linguagens de Programação Imperativas, sendo estas últimas sinônimos das **Linguagens Procedurais**.

Assim, em uma comparação simples, tem-se que:

“um programa é declarativo se descreve o que ele faz e NÃO como seus procedimentos funcionam”.

ORACLE®
PL/SQL

procedural



Indica a parte não procedural



Structured Query Language - SQL

LINGUAGEM

Um sistema de banco de dados relacional, geralmente proporciona dois tipos principais de recursos em sua linguagem SQL:

- a) uma específica para manipular as **estruturas** do BD (DDL);
- b) outra para expressar consultas e atualizações sobre o que é armazenado nessas estruturas (**dados**) - (DML).

- **DDL** - *Data Definition Language*;
- **DML** - *Data Manipulation Language*.



Structured Query Language - SQL

- Linguagem de Definição de Dados (DDL - *Data Definition Language*) – uma estrutura de dados é representada por um conjunto de definições expressas por uma linguagem específica.
 - O resultado no uso da DDL constitui em um arquivo especial chamado de dicionário, catálogo ou diretório de dados;
 - Um dicionário de dados é um arquivo de **metadados**.
- **METADADOS** são dados a respeito de dados. Em um sistema de Banco de Dados, esse arquivo ou diretório é consultado antes que o dado real seja acessado (manipulado).

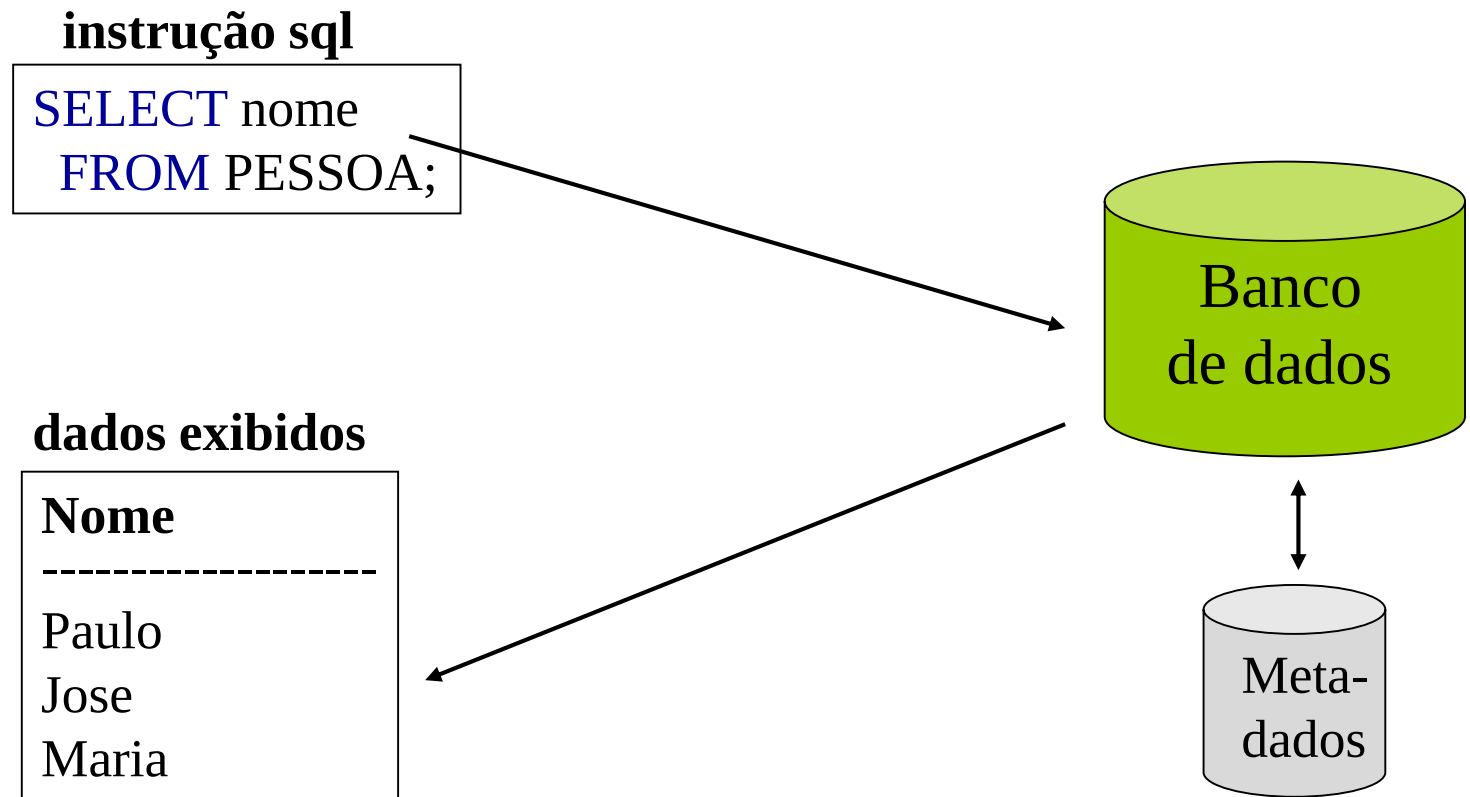


Structured Query Language - SQL

- Linguagem de Manipulação dos Dados (DML - *Data Manipulation Language*) – é a linguagem que viabiliza o acesso ou a manipulação dos dados de forma compatível ao modelo de dados apropriado. Por manipulação de dados entende-se:
 - Recuperação dos dados armazenados no BD;
 - Inserção de novos dados no BD;
 - Remoção ou modificação de dados do BD.
 - Linguagem de Consulta dos Dados (SQL - *Strutured Query Language*) – é parte de uma DML responsável pela recuperação de dados (pesquisa ou consulta).
- Apesar da **SQL** indicar uma linguagem de consulta, ela possui recursos para definição de estruturas de dados (DDL), de manipulação de dados no BD e de definição de segurança.

Structured Query Language - SQL

A SQL consiste em uma linguagem **NÃO procedural** (declarativa) que permite a interface básica para comunicação com o SGBD.



Structured Query Language - SQL

Um SGBD realiza alguns processos que podem ser efetuados por meio da linguagem SQL

- DEFINIÇÃO: criação descritiva do esquema(s) que atenderá as necessidades no BD;
- CONSTRUÇÃO: inserção das instâncias iniciais no banco de dados;
- MANIPULAÇÃO: realização de consultas e atualizações sobre os dados decorrentes do dia a dia que usa a estrutura deste BD.



Structured Query Language - SQL

Será usada algumas ferramentas SQL para manipular o BD. As respectivas ferramentas são baseadas no SQL padrão, mas possuem algumas diferenças, por exemplo, recursos específicos da **ORACLE** que podem ser usados para se escrever relatórios e controlar o modo como a saída de tela ou papel é formatada difere da saída do **MySQL**.

Para começar a usar o BD é necessário estabelecer uma conexão com o SGBD que usa a SQL.

Nas características de um SGBD, lembre-se que existe uma preocupação com a restrição de **acesso** **NÃO autorizado**.



Structured Query Language - SQL

Criando uma Base de Dados

O estabelecimento de uma conexão com o SGBD **MySQL** só pode acontecer com o usuário que esteja cadastrado no BD.

Usando o SGBD **MySQL** será criada uma base de dados específica para a implementação do projeto. Essa base de dados receberá o nome de: ***baseDados***.

DATABASE (base de dados): Conjunto de registros de dados dispostos em estrutura regular que possibilita o seu armazenamento organizado produzindo informação.

CREATE DATABASE <nome da base de dados>;

SHOW DATABASES; -- lista bases de dados (só **MySQL**)

DROP DATABASE <nome da base de dados>;

Structured Query Language - SQL

Estabelecendo a Conexão

Para se conectar ao SGBD **MySQL** disponível no computador local do laboratório use as opções do Sistema Operacional instalado para localizar esse servidor **MySQL** disponível.

Procure a opção que indica o **MySQL** com acesso por linha de (*Command Prompt*) e a execute fornecendo os dados solicitados para um usuário já cadastrado nesse SGBD:

- confirme com seu professor qual o **usuário** disponível;
- solicite ao professor a **senha** individual desse usuário;
- pergunte ao professor qual será a base de dados (*database*) da atividade a ser realizada e execute a instrução abaixo:

use <nome da database>; (só no **MySQL**)

use **baseDados**;

Structured Query Language - SQL

Para iniciar a definição de um SGBD é necessário conhecer os seus tipos de dados válidos.

Os tipos usados na descrição de esquemas têm seus correspondentes em cada SGBD. Alguns do **MySQL** são:

- **int** – conjunto dos números inteiros (ver variações)
- **decimal** (n,d) – conjunto dos números reais
- **char**(n) – de 1 até 255 caracteres
- **varchar**(n) – caracteres variáveis até 4000
- **date** – data com ano, mês e dia no padrão
- **time** – horário com hora, minutos e segundos
 - n corresponde ao comprimento ou tamanho
 - d quantidade de dígitos decimais depois da vírgula

Visite o sítio virtual abaixo e confira todos os tipos do **MySQL**

<https://dev.mysql.com/doc/refman/5.7/en/data-types.html>

Structured Query Language - SQL

A primeira fase em qualquer SGBD começa com o uso de instruções **DDL** (*Linguagem de Definição de Dados*), pois serão por elas criados os recursos no BD.

Exemplo:

→ Como seria a relação que armazenaria todas as unidades da federação brasileira (os estados)?

Supondo esta necessidade elabora-se o esquema coerente para armazenar e manipular essas Unidades da Federação.

Relação: **ESTADO**

sigla literal (2)

nome literal (20)

Nome da relação é no **singular**, mesmo ela podendo guardar vários.

Unidades da Federação

sigla	nome
AC	Acre
AM	Amazonas
:	:
DF	Distrito Federal

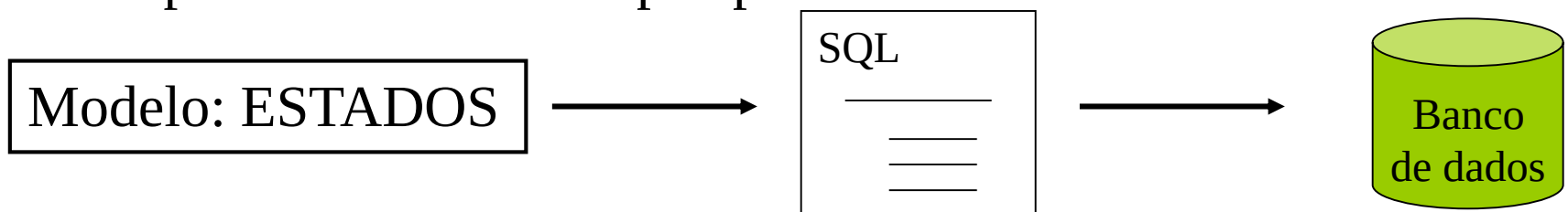
Structured Query Language - SQL

Após modelar e confirmar qual a relação que atende as necessidades, uma tabela será criada no SGBD.

Para criar esta relação, também chamada de **tabela**, será usada a declaração SQL (DDL) que efetiva essa tabela no BD.

```
CREATE TABLE <nome da tabela> (  
  nome do atributo_1 <tipo dado>, -- tipo do atributo_1  
  nome do atributo_2 <tipo dado> ) ;
```

- Na linha de comando do SQL os dois traços seguidos (--) indicam um comentário a partir deles até o final da linha;
- O ponto e vírgula encerra uma instrução, deixando-a pronta para ser executada após pressionada a tecla <ENTER>.



Structured Query Language - SQL

Todos os ESTADOS possuem uma *sigla*, portanto sempre que se for cadastrar um novo estado ele deverá possuir uma *sigla*. Para que o SGBD controle esta restrição na tabela, não permitindo que seja informado um estado sem uma *sigla*, é necessário que o projeto da relação possua uma confirmação que **obrigue** a *sigla*.

Caso nada identifique esta obrigatoriedade, será possível o armazenamento de um estado **SEM** *sigla*.

A obrigatoriedade de um atributo é representada pela expressão **NOT NULL** (não nulo) que deve estar vinculada a todos os atributos que são obrigatórios em uma tabela (veja o exemplo a frente).

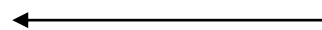


Structured Query Language - SQL

A criação da tabela que representará e armazenará os dados das Unidades da Federação Brasileira será feita com a seguinte instrução SQL (parte DDL):

```
CREATE TABLE ESTADO (  
    sigla CHAR(2) NOT NULL,  
    nome VARCHAR(20) ); -- pressionando <ENTER>
```

Tabela criada.



resultado da operação

Por meio desta instrução (ou comando) SQL foi criada a tabela **ESTADO** que possui como domínio todas as Unidades da Federação Brasileira.



Structured Query Language - SQL

A expressão **NULL** significa que não existe valor, ou seja, não contém dados. O **erro** mais comum que se comete é a atribuição de um valor nulo em um atributo numérico.

O valor **nulo** é diferente de zero, pois zero é um valor, enquanto que **nulo** é a ausência de valor. Se ocorrer a operação $1 + \text{NULL}$ o resultado vai ser **NULL**.

*“Nunca use um **NULL** para representar um valor igual a zero em um atributo numérico. Se este atributo for usado para operações aritméticas dê a ele o valor de zero e não **NULL**”*



Structured Query Language - SQL

No **MySQL** as instruções **DESCRIBE** e **SHOW** são importantes. As duas instruções oferecem um resumo de dados sobre a tabela, também chamada de esquema, com suas colunas (atributos), tipos de dados e outras informações relevantes.

O primeiro comando pode ser usado de maneira abreviada (**4 letras iniciais**), obtendo-se o mesmo efeito.

```
mysql> desc ESTADO;
```

Field	Type	Null	Key	Default	Extra

sigla	char(2)	No		NULL	
nome	varchar(20)	Yes		NULL	

2 rows in set (**0.01 sec**) => tempo de resposta do **SGBD**

Structured Query Language - SQL

A instrução que modifica o nome de uma relação é:

RENAME <nome_atual> **TO** <novo_nome>;

A instrução que remove uma relação (será detalhada a frente) consiste no comando a seguir:

DROP TABLE <nome_da_tabela>;

Exemplo:

RENAME TABLE **ESTADO** **TO** **UNIDADEFEDERAL**;

Tabela renomeada.



resultado

DROP TABLE **ESTADO**;

ERRO 1017: Arquivo não encontrado.



resultado

Código
do erro



Número do erro



Structured Query Language - SQL

Remover uma Tabela

Para se remover uma relação (tabela) do SGBD deve ser usada a instrução **DROP TABLE**.

DROP TABLE <nome da tabela>;

Por meio desta instrução a tabela deixará de existir no BD, sendo todos os dados contidos nessa tabela apagados. Similar a instrução **DROP DATABASE**, que apaga a base de dados com todas as tabelas e recursos desta **base** no SGBD.

Exemplo:

DROP TABLE UNIDADEFEDERAL;

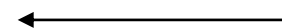
Tabela eliminada.



resultado

DROP TABLE CIDADE;

ERRO 1051: tabela desconhecida.



resultado

Structured Query Language - SQL

Uma importante instrução permite alterar as definições de uma tabela:

ALTER TABLE <nome_da_tabela>

<ação_ser_realizada> <dados_condizentes_ação>;

<ação_ser_realizada> => acrescentar, modificar, retirar recursos da tabela

<dados_condizentes_ação> => dados ou parâmetros coerentes com a ação que alterará a tabela

Exemplo:

Retorne ao *slide 19* e crie novamente a tabela **ESTADO**, depois:

ALTER TABLE **ESTADO** **ADD** pais VARCHAR(100);

Tabela alterada.



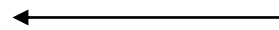
resultado

Structured Query Language - SQL

É possível assim manipular relações (tabelas) com DML, após conhecer os procedimentos básicos para a criação e remoção das relações com comandos DDL.

DROP TABLE ESTADO;

Tabela eliminada.



resultado

Identificando o domínio das unidades da federação, pode-se armazenar seus dados, em que serão guardados os valores de todos os estados da Federação Brasileira.

Com isso, se inicia o processo de construção do BD, pois a definição projetou o BD mais coerente para o armazenamento necessário dos dados do problema.



Structured Query Language - SQL

Instrução INSERT

Esta instrução é usada para inserir tuplas em uma relação.

INSERT INTO <tabela>[(coluna_1,...,coluna_n)] **VALUES** (valor_1,...,valor_n);

Exemplo:

INSERT INTO **ESTADO** (sigla, nome) VALUES('GO', 'Goiás');

ERRO 1146: tabela não existe.

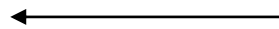


resultado

A tabela desejada **não existe mais**, pois foi apagada pelo comando DROP. Portanto crie a tabela **ESTADO novamente** e depois insira os estados.

INSERT INTO **ESTADO** (sigla, nome) VALUES('GO', 'Goiás');

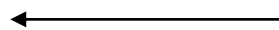
1 linha criada.



Resultado da 1ª forma

INSERT INTO **ESTADO** (nome, sigla) VALUES('ACRE', 'AC');

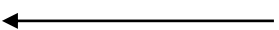
1 linha criada.



Resultado da 2ª forma

INSERT INTO **ESTADO** VALUES('SP', 'São Paulo');

1 linha criada.



Resultado da forma geral

Structured Query Language - SQL

A instrução INSERT deve respeitar as características de cada tipo de dado para inserir dados na base de dados corretamente nas formas exigidas para cada tipo de dado no **SQL**.

Um exemplo relevante é a manipulação correta de caracter(es) ou *String* em **SQL** que deve ser definida usando **APÓSTROFE** e **NÃO ASPAS**. Usa-se **ASPAS** somente em situações muito específicas e bem definidas em **SQL**, pois é gerado um objeto com tratamento específicos e dispendiosos quando não se é preciso usar as **ASPAS**.

```
INSERT INTO ESTADO VALUES  "SE", "Sergipe");
```

```
INSERT INTO ESTADO VALUES('SE', 'Sergipe');
```

1 linha criada.

← resultado

Structured Query Language - SQL

Instrução DELETE

Esta instrução é usada para remover uma ou mais tuplas da relação (tabela), possuindo duas formas básicas:

DELETE FROM <tabela>;

ou

DELETE FROM <tabela> **WHERE** <condição>;

A primeira forma é obrigatória e apaga a relação e todos os seus dados, enquanto que a segunda possui uma parte opcional, a partir da cláusula **WHERE**, que apaga somente os dados da tabela que atendem a uma condição (ou condições) imposta por essa cláusula (**WHERE**).



Structured Query Language - SQL

Exemplo:

DELETE FROM ESTADO WHERE sigla = 'SP';

1 linha deletada.

← resultado

DELETE FROM ESTADO WHERE nome = 'ACRE';

1 linha deletada.

← resultado

DELETE FROM ESTADO WHERE sigla = 'AC';

0 linhas deletadas.

← resultado

INSERT INTO ESTADO VALUES('DF', 'BRASÍLIA');

1 linha criada.

← resultado

DELETE FROM ESTADO;

2 linhas deletadas.

← resultado



Structured Query Language - SQL

Instrução UPDATE

Esta instrução permite a atualização de um ou mais atributos em uma tupla, ou em um grupo de tuplas de uma relação (tabela). O conteúdo de cada atributo será ajustado com a cláusula **SET**. Como na instrução DELETE existem duas formas para UPDATE:

UPDATE <tabela> **SET** <atributo> = <valor>;

ou

UPDATE <tabela> **SET** <atributo> = <valor>

WHERE <condição>;

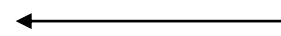
A primeira forma é obrigatória e atualiza todos os dados da relação, enquanto que a segunda possui uma parte **WHERE** opcional, que atualiza somente os dados da relação que atendem a uma condição (ou condições) imposta pela cláusula **WHERE**.

Structured Query Language - SQL

Exemplo:

UPDATE ESTADO SET nome = 'ACRE';

0 linhas atualizadas.

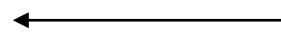


resultado

Com a remoção de todas as tuplas no último **DELETE**, não é possível alterar nenhum dado, pois não existem dados na tabela.

INSERT INTO ESTADO VALUES('BA', 'BAIA');

1 linha criada.

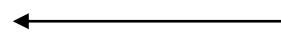


resultado

UPDATE ESTADO SET nome = 'Bahia'

WHERE sigla = 'BA';

1 linha atualizada.

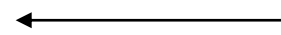


resultado

UPDATE ESTADO SET nome = 'ALAGOAS',

sigla = 'AL' WHERE sigla = 'PA';

0 linhas atualizadas.



resultado



Structured Query Language - SQL

Exemplo:

INSERT INTO ESTADO VALUES('PA', 'PARA');

1 linha criada.

← resultado

UPDATE ESTADO SET nome = 'ACRE'

WHERE sigla = 'PA';

1 linha atualizada.

← resultado

UPDATE ESTADO SET nome = 'ALAGOAS',

sigla = 'AL' WHERE sigla = 'PA';

1 linha atualizada.

← resultado

UPDATE CIDADE SET nome = 'BRASÍLIA'

ERRO 1146: tabela não existe.

← resultado

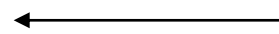


Structured Query Language - SQL

Realizando algumas atualizações no banco de dados:

```
UPDATE ESTADO SET nome = 'BRASIL'
      WHERE nome = 'TOCANTINS';
```

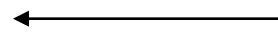
0 linhas atualizadas.



resultado

```
UPDATE ESTADO SET nome = 'BRASIL'
      WHERE nome = 'Bahia';
```

1 linha atualizada.

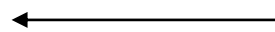


resultado

ATENÇÃO que alguns SGBDs respeitam o *case sensite*, de acordo com seu S.O. e sua própria configuração de instalação.

```
UPDATE ESTADO SET nome = 'CEARA',
      sigla = 'CE' WHERE sigla = 'BA';
```

1 linha atualizada.



resultado

Structured Query Language - SQL

Instrução SELECT

Esta instrução é a essência do linguagem SQL. É por meio dela que se recupera (pesquisa) dados de um banco de dados. De modo simples, está se dizendo ao SGBD quais informações foram selecionadas para serem recuperadas.

Pode-se dividir esta instrução em quatro partes básicas:

- **SELECT** – seguido de atributos que se esperam ver (**obrigatório**)
- **FROM** – seguido da origem dos dados no BD (**obrigatório**)
- **WHERE** – seguido das restrições de recuperação (**opcional**)
- **ORDER BY** – seguido da forma como os dados serão ordenados (**opcional**)

Esta será a instrução mais comumente usada na linguagem SQL.

Structured Query Language - SQL

O símbolo asterisco (*) significa que todos atributos da relação informada deverão ser recuperados.

Exemplo: SELECT * FROM ESTADO;

```
-----  
| sigla | nome      |  
-----  
| BA    | Bahia     |  
| AL    | ALAGOAS  |  
-----
```

2 linhas apresentadas.

← resultado

SELECT * FROM CIDADE;

ERRO 1146: Tabela não existe.

← resultado

DELETE FROM ESTADO;

2 linhas deletadas.

← resultado

Structured Query Language - SQL

Consulta sem dados na relação e a inserção de dados:

```
SELECT * FROM ESTADO;
```

não há linhas selecionadas.

← resultado

```
INSERT INTO ESTADO (sigla, nome) VALUES('PA', 'PARA');
```

1 linha criada.

← resultado

```
INSERT INTO ESTADO (nome, sigla) VALUES('Parana','PR');
```

1 linha criada.

← resultado

```
INSERT INTO ESTADO VALUES('AM','AMAZONAS');
```

1 linha criada.

← resultado

```
SELECT nome FROM ESTADO;
```

← resultado

nome
PARA
Parana
AMAZONAS

3 linhas apresentadas.

Atenção

O uso do * compromete a performance SGBD.

Structured Query Language - SQL

Pesquisando as informações atuais no SGBD tem-se:

SELECT sigla , nome **FROM ESTADO**;

sigla	nome
PA	PARA
PR	Parana
AM	AMAZONAS

Especificando as colunas desejadas na ordem em que elas serão procuradas e apresentadas

A consulta de somente algumas tuplas pode ser conseguida por meio da cláusula **WHERE**. O uso desta cláusula é **opcional** e faz com que uma ou mais condições tenham que ser atendidas para que uma tupla seja recuperada e apresentada (**projetada ao usuário**).



Structured Query Language - SQL

A forma básica de uma instrução **SELECT** pode ser generalizada em:

SELECT <atributos> **FROM** <tabela> **WHERE** <condições>;

Após o **WHERE** são especificadas as cláusulas condicionais que restringirão as tuplas a serem recuperadas pela consulta elaborada.

SELECT nome **FROM** ESTADO

WHERE sigla = 'PR';

nome



resultado

Parana

UPDATE ESTADO **SET** nome = 'BRASIL'

WHERE nome = 'AMAPA';

0 linhas atualizadas.



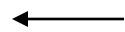
resultado



Structured Query Language - SQL

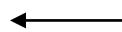
```
UPDATE ESTADO SET nome = 'PARA'
      WHERE sigla = 'AM';
```

1 linha atualizada.



resultado

```
SELECT * FROM ESTADO;
sigla nome
```



resultado

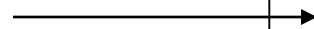
```
----
PA  PARA
PR  Parana
AM  PARA
```

```
SELECT nome, sigla FROM ESTADO
      WHERE nome = 'PARA';
```

```
nome      sigla
-----
PARA      PA
PARA      AM
```



resultado



```
SELECT * FROM ESTADO
WHERE nome = 'PARA' AND sigla = 'AM';
      sigla  nome
-----
AM        PARA
```

Structured Query Language - SQL

Na cláusula **WHERE** serão utilizados alguns operadores de comparação e lógicos para que a condição seja elaborada e possa ser constatada com mais realidade.

Relacional (comparação)	
Oper.	Significado
=	igual a
>	maior que
<	menor que
>=	maior ou igual que
<=	menor ou igual que

Lógico	
Oper.	Significado
AND	E lógico
OR	OU lógico
NOT	Negação

```
SELECT nome FROM ESTADO
```

```
WHERE sigla = 'SP' OR nome = 'Parana';
```

```
nome
```

```
-----
```

```
Parana
```



resultado

Structured Query Language - SQL

A cláusula das outras instruções de manipulação da informação também atendem as características dos operadores lógicos e de comparação. Então, corrija as outras duas tuplas que aparecem com o nome de 'BRASIL' para o correto nome, de acordo com a sua *sigla* já cadastrada.

```
UPDATE ESTADO SET nome = 'PARANA'
```

```
WHERE sigla = 'PR';
```

1 linha atualizada.

← resultado

```
UPDATE ESTADO SET nome = 'AMAZONAS'
```

```
WHERE sigla = 'AM';
```

1 linha atualizada.

← resultado

Agora, recupere somente as tuplas que sofreram alteração e verifique se elas estão corretas.

```
SELECT * FROM ESTADO
```

```
WHERE sigla = 'PR' OR sigla = 'AM';
```

resultado →

sigla	nome
-----	-----
PR	PARANA
AM	AMAZONAS

Structured Query Language - SQL

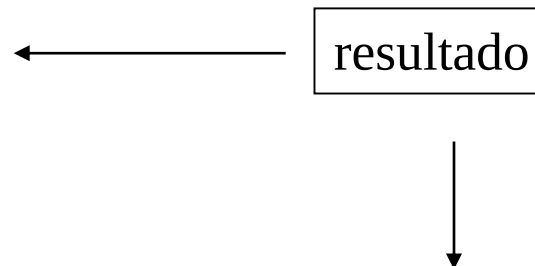
A apresentação dos dados recuperados pelo **SELECT** pode ser classificada (ordenada) de forma **ascendente** (crescente) ou **descendente** (decrescente).

Para isso se pode incluir a cláusula **opcional ORDER BY** no final da instrução **SELECT**.

```
SELECT nome FROM ESTADO  
ORDER BY nome ASC;
```

nome

AMAZONAS
PARA
PARANA



```
SELECT nome FROM ESTADO  
ORDER BY nome DESC;
```

nome

PARANA
PARA
AMAZONAS

Structured Query Language - SQL

Uma classificação com mais que um atributo pode ser feita com a especificação ordenada de quais os atributos a serem usados na classificação e a ordem de prioridade para cada um.

Os atributos devem ser especificados separados por vírgula e devem estar em ordem de prioridade, conforme sua especificação na cláusula **ORDER BY**, em que o primeiro atributo especificado será classificado primeiramente e o segundo, respeitando a classificação do primeiro, será classificado em seguida, dentro da primeira classificação.

Exemplo:

```
SELECT nome FROM EMPREGADO WHERE salario < 0  
ORDER BY nome, salario; →
```



Primeira classificação será feita por **nome** e dentro da classificação por **nome** será feita por **salário**

Sem a especificação de **ASC** ou **DESC** a consulta é classificada de forma **ASC**, por definição (padrão)

Structured Query Language - SQL

Diante do avanço no estudo das instruções SQL, e vislumbrando a eficiência e agilidade dos profissionais envolvidos em trabalhar sobre as instruções SQL são empregadas *boas práticas* na elaboração dessas instruções, por exemplo:

```
SELECT idEmpregado, nome
FROM EMPREGADO
WHERE salario < 0
ORDER BY nome, salario;
```



Note que a instrução acima possui cada uma de suas cláusulas alinhadas, estando a primeira palavra-chave da cláusula de instrução alinhada ao seu final (*à direita*).



Structured Query Language - SQL

Outra alternativa no uso do *Workbench* do **MySQL**, mas também interessante na qualidade, é o uso da tecla de atalho

Ctrl + B

```
SELECT
    idEmpregado, nome
FROM
    EMPREGADO
WHERE
    salario < 0
ORDER BY nome, salario;
```

Com isso, a visão para compreensão da lógica envolvida e da álgebra proposta se torna mais fácil e rápida de ser compreendida aos profissionais da área de TI em **qualquer instrução SQL**.



Structured Query Language - SQL

Baseado no exemplo anterior, observe o comando DDL que criaria uma relação compatível com a consulta solicitada pelo ***SELECT*** anteriormente proposto.

```
CREATE TABLE EMPREGADO (  
  idEmpregado      INT(8) NOT NULL,      -- código funcional  
  nome             VARCHAR(40) NOT NULL,  -- nome do empregado  
  dtNascer         DATE NOT NULL,         -- data de nascimento  
  salario          DECIMAL(7.2) NOT NULL, -- valor do seu salário  
  cpf              INT(11) NULL -- número do CPF, se tiver (não é obrigatório)  
);
```

Padrões que indicam significados nos atributos (há outros)

Identificador único => **id**Empregado (para identificadores)

Data de nascimento => **dt**Nascer (para datas)

Descrição do carro => **ds**Carro (envolve caracteres)

Exercícios de Fixação

1.) Com o objetivo de recordar, ou praticar para quem ainda não trabalhou com as **boas práticas** na implementação correta de uma base de dados, resolva este exercício elaborando os *scripts* **Fisico**, **Popula** e **Apaga** como exige a disciplina.

→ Crie as 2 relações a seguir, respeitando as descrições:

RELAÇÃO ESTADO

sigla CHARACTER(2)

nome CHARACTER(20)

RELAÇÃO CIDADE

codigo NUMÉRICO(5)

nome CHARACTER(50)

sigla CHARACTER(2)

habitantes NUMÉRICO

→ Agora, insira **4 tuplas** válidas na relação ESTADO e mais **7 tuplas** na relação CIDADE, em que ao menos 1 tupla seja de cada estado cadastrado na relação ESTADO e 1 estado fique sem nenhuma cidade cadastrada associado a ele.

Exercícios de Fixação

- Garanta que sua base de dados NÃO ACEITARÁ nome de cidades iguais para um mesmo ESTADO no projeto.
- Realize algumas CONSULTAS sobre estes dados, sendo um *SELECT* para cada solicitação abaixo:
 - A) Projeção de sigla e nome do estado de **MS** e **DF**, sendo mostrado uma vez só se tiverem repetidas na tabela de cidades;
 - B) Selecione só o nome das cidades e a sigla dos estados de **DF**, **GO** e **MS** sobre as cidades em ordem alfabética do estado e em cada estado a ordem por nome de cidade;
 - C) Selecione todas as cidades do **primeiro** estado cadastrado mostrando só o nome da cidade, nome e sigla do estado;
 - D) Consulte o nome e a sigla do estado cadastrado por **último** e todas as cidades cadastradas nele, mostrando o nome e a quantidade de habitantes da cidade em ordem alfabética desse estado.

Structured Query Language - SQL

Além dos comandos DDL e DML vistos e revisados até aqui, uma outra categoria de comandos chamada de **TPL** (*Transaction Proccess Language* – Linguagem de Processamento de **Transação**) compõe a SQL.

Os comandos desta categoria são responsáveis pela gravação definitiva no banco de dados das instruções DDL e DML, além da recuperação do banco de dados, caso algo **errado** aconteça.

Várias das instruções DDL são de salvamento automático (***autocommit***), portanto, uma vez executadas elas já estão implantadas no SGBD. Por exemplo a instrução **CREATE TABLE**.



Structured Query Language - SQL

Salvando as Realizações

Após realizar algumas instruções sobre o banco de dados é necessário solicitar que ele armazene efetivamente (**grave**) o resultado de suas solicitações feitas por meio das instruções executadas (**transações**).

A efetiva realização solicitada pelas instruções **INSERT**, **DELETE** e **UPDATE** precisam ser salvas no banco de dados. Para que as suas ações (ou realizações) sejam salvas no banco de dados, é necessária a execução do comando **COMMIT**.

Por meio do comando **COMMIT** essas instruções DML são efetivadas (**salvas**) no banco de dados.



Structured Query Language - SQL

Salvando as suas realizações no SGBD será liberado todo recurso exigido até concluir a sua transação. Existe uma redução nos "**custos**" do acesso de outros usuários e a diminuição do seu tempo de espera.

Outra característica importante para estar salvando sempre o que você estiver realizando, principalmente as DML, **diminui os riscos de perder** algo que terminou de fazer com tanto cuidado no SGBD.

Exemplo:

Após uma instrução DML correta ser executada salve-a com:

COMMIT;

Efetivação completada.



resultado



Structured Query Language - SQL

Durante o uso do SGBD pode acontecer a execução de uma instrução equivocada, ou problemas como a falta de energia no meio de uma transação, entre outros.

A restauração da situação do SGBD, antes de um salvamento acontecer, é possível com **ROLLBACK**. Essa outra instrução SQL o retrocede para situação que estava após o último salvamento (**COMMIT**) ser executado.

Exemplo:

ROLLBACK;

Restabelecimento completado.

← resultado

As instruções de **AUTOCOMMIT** não serão retornadas, pois assim que são executadas são salvas, **PERSISTINDO** no BD.

Vários SGBD permitem configurar a ação de **AUTOCOMMIT**.

Structured Query Language - SQL

Atributo de Auto Incremento na Tabela

A implementação de um atributo (coluna da tabela) de **auto incremento** permite que um número inteiro único seja gerado quando um novo registro for inserido no **MySQL**, nunca sendo repetido este número.

A palavra ou expressão chave **AUTO_INCREMENT** deve estar indicada em um único atributo da tabela. Seu valor inicial padrão é 1 e o incremento será de 1 em 1.

Porém, é possível definir um valor inicial diferente do padrão, assim como alterar o valor do auto incremento (**o passo**) para seguir a partir de um valor arbitrário que será indicado por uma instrução DDL (***alter table***).

Structured Query Language - SQL

Características do Atributo de Auto Incremento

- Ao inserir novos valores na tabela, não é necessário especificar o valor do atributo de **auto incremento**;
- Só é permitido usar um atributo de auto incremento por tabela, geralmente do tipo inteiro;
- Necessita também da restrição (*constraint*) **NOT NULL**, que será configurado automaticamente, mas deve ser incluída na documentação e no *script* que cria a tabela e o atributo corretamente;
- Exige criação da chave primária (**PK**) no atributo.



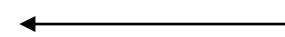
Structured Query Language - SQL

Exemplo:

```
CREATE TABLE EQUIPE (  
    idEquipe INT(5) NOT NULL AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    CONSTRAINT equipe_ PK PRIMARY KEY(idEquipe)  
) ENGINE = InnoDB AUTO_INCREMENT = 1 ;
```

```
INSERT INTO EQUIPE (nome) VALUES ('Flamengo');
```

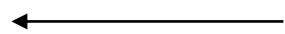
1 linha criada.



resultado

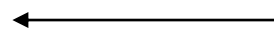
```
INSERT INTO EQUIPE VALUES (null, 'Vasco');
```

1 linha criada.



resultado

```
SELECT * FROM EQUIPE;
```



conferir seu resultado

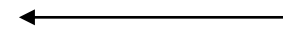


Structured Query Language - SQL

Exemplo:

INSERT INTO EQUIPE VALUES('Fluminense');

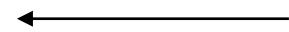
1 linha criada.



resultado

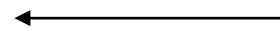
INSERT INTO EQUIPE VALUES(7, 'Botafogo');

1 linha criada.



resultado

SELECT * FROM EQUIPE;



conferir seu resultado

INSERT INTO EQUIPE (nome) VALUES('Palmeiras');

1 linha criada.

Qual resultado (valor) será apresentado no atributo **idEquipe** dessa última inserção (Palmeiras)?



Structured Query Language - SQL

Outros bancos de dados (SGBD) empregam outras formas de controle sobre valores únicos e sequenciais relevantes as implementações relacionais em SGBDs.

Por exemplo:

ORACLE utiliza os objetos de banco de dados conhecidos como **SEQUENCE**;

PostgreSQL usa também a **SEQUENCE**, mas com alguns outros recursos e sintaxes diferentes do **ORACLE**.



Structured Query Language - SQL

DOCUMENTANDO SCRIPT (cabeçalho)

A elaboração de qualquer *script* de banco de dados exige uma documentação mínima **OBRIGATÓRIA** contendo:

- Identificação de qual projeto o *script* pertence;
- Banco de dados e base(s) de dados envolvidas;
- Autor ou autores do *script*;
- Data de criação e de alterações do *script*;
- Relato de cada alteração realizada no *script*;
- Síntese de todos os objetos e recursos de SGBD que fazem parte do projeto que o *script* pertence.



Structured Query Language - SQL

Exemplo do **cabeçalho obrigatório** que documenta um *script*:

```
-- ----- Acompanhamento Escolar -----
```

Identificação do projeto

```
--      SCRIPT DE CRIACAO (DDL)
```

Dados fundamentais do *script*

```
-- Data Criacao .....: 21/03/2018
-- Autor(es) .....: Ana Maria Braga
-- Banco de Dados .....: MySQL 5.7
-- Base de Dados (nome) ...: bdEscolar
--
```

```
-- PROJETO => 01 Base de Dados
--           => 03 Tabelas
--           => 02 Perfis de Usuario
--
```

Síntese dos objetos de BD existentes no projeto

```
-- Ultimas Alteracoes
-- 28/04/2018 => Criacao da nova tabela TURMA;
--           => Incluída propriedade AUTO_INCREMENT na tabela CURSO;
-- 30/04/2018 => Ajuste complementar na documentacao deste script;
-- -----
```

Alterações no *script* e no BD

```
CREATE DATABASE bdEscolar;
:
```

Todos comandos SQL contidos no *script* conforme a demanda do que foi solicitado.

Structured Query Language - SQL

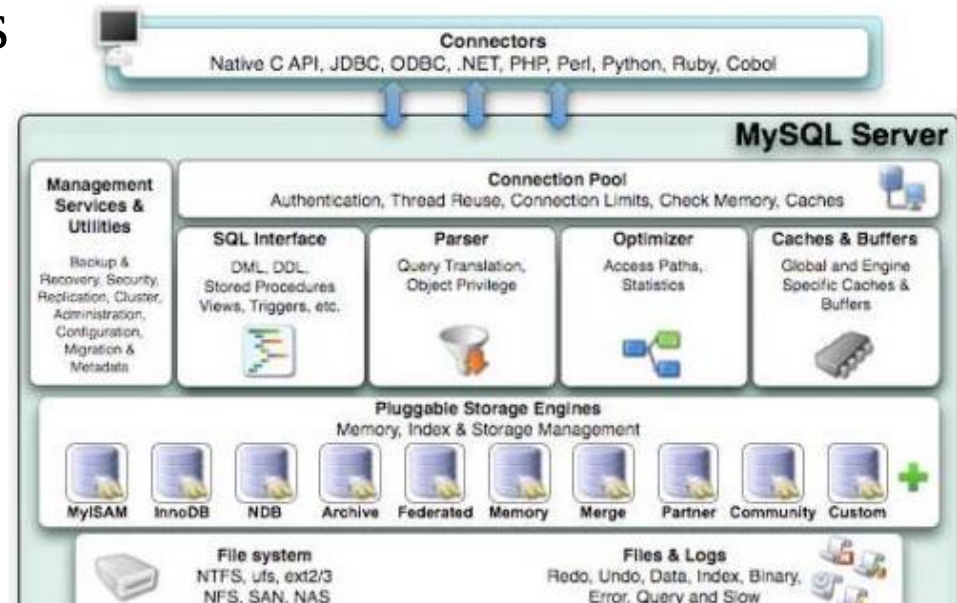
Opção **ENGINE** no **MySQL**

A **Engenharia de Armazenamento** (*Storage Engine*) confere características modulares que podem ser atribuídas as relações (tabelas) de uma base de dados no **MySQL**.

Essas características, também chamadas de Motores de Armazenamento, são nativas do **MySQL** (**InnoDB**, **CSV**, **Federated**, **MyISAM**, **Memory**, **Archive**), podendo outras personalizadas serem integradas a este SGBD.

Consulte para saber mais ↓

<https://medium.com/@claudioldf/diferen%C3%A7as-entre-as-storage-engines-do-mysql-myisam-innodb-e-outras-1dda5e3bac05>



Structured Query Language - SQL

Exemplo:

Suponha que as tabelas criadas em outros SGBD sejam sempre ***Ferraris***, ou seja, tenham potente motor, grande velocidade e outros apetrechos de luxo.

Imagine também que exista a necessidade de cruzar uma avenida bem congestionada e você deverá escolher entre os veículos disponíveis qual será o mais rápido:

uma Ferrari ou uma (excelente) Bicicleta



No **MySQL** é possível escolher qual motor (*storage engine*) será usado de maneira coerente com a necessidade da atividade que será realizada. Assim, pode-se dirigir uma Ferrari (**tabelas transacionais**) ou pedalar uma bicicleta (**não transacionais**), conforme seja mais conveniente ao SGBD.

Structured Query Language - SQL

Quais são as principais características, recursos ou funcionalidades inerentes as **Engenharias de Armazenamento** (*Storage Engines*) do **MySQL**:

- a) Capacidade Transacional: capacidade da tabela aceitar múltiplos acessos (de múltiplos usuários/aplicações), com colisão e travamento mínimos, sem que um usuário interfira com a operação do outro, além das propriedades **ACID**;
- b) Meio de armazenamento: no **MySQL** a forma de armazenamento e uso das tabelas depende do motor escolhido;
- c) Índices: de acordo com o motor têm-se diferentes tipos de índices;
- d) Integridade Referencial: há motores que usam e outros que não usam e/ou respeitam a chave estrangeira (*foreign key*);



Structured Query Language - SQL

- e) Tipo de bloqueio: capacidade de poder bloquear o acesso a um único registro (linha), vários registros ou a tabela inteira;
- f) Cópia de segurança *on line* (BOL – *Backup On Line*): realizar *backup* sem parar o trabalho do BD e de seus usuários;
- g) Reparação Automática (*Auto Recovery*): havendo falha no banco de dados (BD) alguns motores registrarão no LOG do **erro** que foi encontrado e consertado automaticamente.

Em resumo, *storage engine* (motor) não é simplesmente um tipo de tabela, mas características únicas e exclusivas do **MySQL** que permitem escolher um conjunto de recursos pré-definidos para melhor atender aos requisitos e as necessidades **DE CADA TABELA** em sua base de dados, respeitando também o hardware disponível.



Exercícios de Fixação

2.) Faça a revisão de sua proposta de solução para o Exercício 1 desta aula (**Aula 3**) e corrija a documentação OBRIGATÓRIA exigida por esta disciplina (**cabeçalho**) nos 3 *scripts* **Fisico**, **Popula** e **Apaga** exigidos pelo enunciado e defina para cada tabela qual a Engenharia de Armazenamento (**ENGINE**) para transações seguras e respeito pleno às regras de integridade como as chaves estrangeiras (*FKs*) e outras.

A partir desta revisão e possíveis ajustes para o trabalho contínuo e consciente do uso das **boas práticas** em BD, inclua em **uma nova instrução** específica a tentativa de inserir **uma nova** CIDADE que seja de um ESTADO que **NÃO** está cadastrado ainda. Como comentário de *script* SQL informe o código de erro gerado e o seu significado completo em português logo abaixo dessa nova instrução DML que estará no fim do **Popula**.

Exercícios de Fixação

- 3.) A Secretaria de Saúde resolveu contratar uma equipe de desenvolvedores de software para solucionar um problema de gerenciamento nas Escalas de Plantonistas em um hospital. O hospital é composto por vários setores e cada setor possui um nome (**ou** a descrição do nome) específico. Cada plantonista está alocado para um setor em um horário determinado. O registro dos plantonistas possui nome completo, matrícula funcional (só numérica e inteira) e sexo (código com caractere único **M** ou **F**) que precisam ser armazenados, sendo relevantes tais informações para sua melhor identificação e atribuições no hospital. Cada plantonista ainda pode possuir alguma especialidade médica ou não, sendo assim um **Enfermeiro Geral** se não possuir nenhuma especialidade médica em sua formação.

Exercícios de Fixação

... continuação do exercício 3

Um plantonista pode estar alocado para mais de um setor, porém não no mesmo horário. Cada setor pode ter vários plantonistas em um mesmo horário, independente da sua especialidade, se possuir alguma. Cada setor é dividido por alas, sendo nelas definidos quantos quartos (**no mínimo 1**), salas de cirurgia (**pode não existir**) e de enfermaria (**no mínimo 1**) estão disponíveis, além da quantidade de profissionais da área de Saúde que atuam regularmente em cada ala (**no mínimo 1 por ala**). Um setor é constituído por ao menos uma ala, mas pode ter mais. Porém, uma ala só pertence a um setor e não existe ala sem setor.

Elabore um projeto lógico de banco de dados com **ME-R**, **DE-R**, **DLD** e 3 *scripts* básicos) no SGBD **MySQL**.

Exercícios de Fixação

... continuação do exercício 3

O projeto deve atender essa situação e fornecer o acompanhamento adequado ao hospital sobre seus plantonistas e cadastros adequados, respeitando as suas restrições e trazendo a integridade para esta base de dados respeitando até a Terceira Forma Normal (**3FN**).

Faça a modelagem completa a partir do **ME-R** e o **Dicionário de Dados** do projeto para esse problema construindo uma proposta de implementação da base de dados relacional adequada às necessidades de acompanhamento desse hospital usando para isso o **brModelo** na versão 3.32 ou superior como ferramenta *CASE* para gerar os 3 níveis coerentes ao Projeto Lógico de Banco de Dados (com níveis Conceitual, Lógico e Físico), que consistirá na

Exercícios de Fixação

... continuação do exercício 3

sua proposta de completa solução. Lembro da necessidade do arquivo de documentação (**DOC**) que é o responsável pela análise de sua atividade.

Se ele estiver ausente na entrega do projeto **NÃO** será considerado desenvolvido a contento o que foi solicitado, pois é através do arquivo **DOC** que a proposta é analisada e corrigida.

A partir do estudo de revisão desta aula (**Aula 3**), ou de novidade aos estudantes que ainda não conheciam o SGBD **MySQL**, a definição da Engenharia de Armazenamento (**ENGINE**) passa ser obrigatório na elaboração de projetos de banco de dados com qualidade neste SGBD.

Referência de Criação e Apoio ao Estudo

Material para Consulta e Apoio ao Conteúdo

- ELMASRI, R. e Navathe, S. B., Fundamentals of Database Systems, Addison-Wesley, 3rd edition, 2000
 - Capítulo 8
- SILBERSCHATZ, A. & Korth, H. F., Sistemas de Banco de Dados - livro
 - Capítulo 4
- Universidade de Brasília (UnB Gama - **CAE**)
 - <https://sae.unb.br/cae/conteudo/unbfga>
(escolha a disciplina **Sistemas de Banco de Dados 1**)

